

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skilling Session 7: Vector Databases - Foundations

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO2
Session	Skilling 7	Date	

Aim

To understand why a traditional RDBMS index fails for similarity search (curse of dimensionality), what embeddings are, how cosine similarity, dot product and Euclidean distance differ, and how exact search compares with approximate nearest neighbour (ANN) indexes HNSW and IVF.

Program

vector_foundations.py

```
"""
Skilling 7: Vector Database Foundations.
Embeddings, similarity metrics, the curse of dimensionality, exact vs approximate
nearest-neighbour search, and the HNSW / IVF index families.

Run: pip install numpy scikit-learn faiss-cpu
     python vector_foundations.py
"""
import time

import faiss
import numpy as np
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer

rng = np.random.default_rng(42)
np.set_printoptions(precision=3, suppress=True)

def section(title):
    print(f"\n{'=' * 70}\n{title}\n{'=' * 70}")

# -----
# 1. Why an RDBMS B-tree cannot do similarity search: curse of dimensionality
# -----
section("1. Curse of dimensionality: nearest and farthest points look alike")
print(f"{'dims':>6} | {'nearest':>8} | {'farthest':>8} | farthest / nearest")
for dims in (2, 10, 100, 1000):
    points = rng.random((2000, dims))
    query = rng.random(dims)
    d = np.linalg.norm(points - query, axis=1)
    print(f"{'dims':>6} | {d.min():8.3f} | {d.max():8.3f} | {d.max() / d.min():8.2f}")
print("As dimensions grow the ratio tends to 1, so 'close' vs 'far' loses meaning and\n"
      "range-based indexes (B-trees) cannot prune the search space.")

# -----
# 2. Embeddings: dense vectors that capture meaning
# -----
section("2. Text embeddings (dense vectors) for Task Hub task titles")
docs = [
    "Fix login bug in the authentication service",
    "Add JWT token authentication to the login API",
    "Hash user passwords and secure the login token",
    "Design database schema for tasks and users",
    "Write SQL queries and indexes for the task database",
    "Normalise the database tables for users and projects",
    "Create dark mode theme for the dashboard UI",
    "Improve CSS layout of the mobile dashboard UI",
    "Add charts to the project dashboard UI",
]

# LSA embeddings: TF-IDF followed by SVD gives a dense, low-dimensional vector per text.
# In production the same role is played by a neural model, e.g.
```

```

# from sentence_transformers import SentenceTransformer
# model = SentenceTransformer("all-MiniLM-L6-v2") # 384 dimensions
# vectors = model.encode(docs, normalize_embeddings=True)
# or OpenAI "text-embedding-3-small" (1536 dimensions).
tfidf = TfidfVectorizer(stop_words="english")
svd = TruncatedSVD(n_components=3, random_state=0)
doc_vectors = svd.fit_transform(tfidf.fit_transform(docs))
doc_vectors /= np.linalg.norm(doc_vectors, axis=1, keepdims=True)
for text, vec in zip(docs, doc_vectors):
    print(f"{np.array2string(vec, precision=2, sign=' '):>22} {text}")

query = "security issue with user login token"
q_vec = svd.transform(tfidf.transform([query]))[0]
q_vec /= np.linalg.norm(q_vec)
scores = doc_vectors @ q_vec
print(f"\nQuery: '{query}' -> vector {q_vec}")
for rank, i in enumerate(np.argsort(-scores)[:3], 1):
    print(f" {rank}. cosine={scores[i]:.3f} {docs[i]}")

# -----
# 3. Similarity metrics
# -----
section("3. Similarity metrics: cosine, dot product, Euclidean distance")
a = np.array([1.0, 2.0, 3.0])
b = np.array([2.0, 4.0, 6.0]) # same direction as a, twice the length
c = np.array([3.0, 2.0, 1.0]) # different direction

def cosine(x, y):
    return float(x @ y / (np.linalg.norm(x) * np.linalg.norm(y)))

print(f"{'pair':>6} | {'cosine':>7} | {'dot':>6} | {'euclidean':>9}")
for name, x, y in (("a,b", a, b), ("a,c", a, c)):
    print(f"{name:>6} | {cosine(x, y):7.3f} | {x @ y:6.1f} | {np.linalg.norm(x - y):9.3f}")
print("cosine : direction only (a and b are identical in meaning) - use for text embeddings\n"
      "dot : direction AND magnitude - equals cosine when vectors are normalised\n"
      "euclidean: straight-line distance - use when absolute magnitude matters")

# -----
# 4. Exact (brute-force) search vs approximate nearest neighbour (ANN)
# -----
section("4. Exact kNN vs ANN on 100,000 vectors of 128 dimensions (k = 10)")
N, DIM, K, QUERIES = 100_000, 128, 10, 200
# Real embeddings form clusters of related meaning, so the data is generated
# as 500 topic centres with points scattered around each centre.
centres = rng.standard_normal((500, DIM)).astype("float32") * 4
data = (centres[rng.integers(0, 500, N)] + rng.standard_normal((N, DIM))).astype("float32")
queries = (centres[rng.integers(0, 500, QUERIES)] + rng.standard_normal((QUERIES, DIM))).astype("float32")

def timed_search(index, label, build_seconds=None):
    start = time.perf_counter()
    _, ids = index.search(queries, K)
    per_query_ms = (time.perf_counter() - start) * 1000 / QUERIES
    recall = np.mean([len(set(ids[i]) & set(truth[i])) / K for i in range(QUERIES)])
    build = f"{build_seconds:6.2f}s" if build_seconds is not None else " - "
    print(f"{label:>34} | build {build} | {per_query_ms:7.3f} ms/query | recall@10 = {recall:.3f}")
    return ids

exact = faiss.IndexFlatL2(DIM) # exact search: compares with every vector
exact.add(data)
_, truth = exact.search(queries, K)
timed_search(exact, "Flat (exact brute force)")

# IVF: k-means splits the space into nlist cells; a query only scans nprobe cells
quantizer = faiss.IndexFlatL2(DIM)
ivf = faiss.IndexIVFFlat(quantizer, DIM, 256)
t0 = time.perf_counter(); ivf.train(data); ivf.add(data); ivf_build = time.perf_counter() - t0
for nprobe in (1, 8, 32):
    ivf.nprobe = nprobe
    timed_search(ivf, f"IVF (nlist=256, nprobe={nprobe})", ivf_build if nprobe == 1 else None)

# HNSW: multi-layer proximity graph; search greedily descends from the top layer
hnsw = faiss.IndexHNSWFlat(DIM, 32) # M = 32 links per node
hnsw.hnsw.efConstruction = 80
t0 = time.perf_counter(); hnsw.add(data); hnsw_build = time.perf_counter() - t0

```

```

for ef in (16, 64, 256):
    hnsw.hnsw.efSearch = ef
    timed_search(hnsw, f"HNSW (M=32, efSearch={ef})", hnsw_build if ef == 16 else None)

print("\nTrade-off: ANN indexes give up a little recall for a large speed-up. Increasing\n"
      "nprobe (IVF) or efSearch (HNSW) raises recall at the cost of query time.")
print(f"HNSW graph has {hnsw.hnsw.max_level + 1} layers; "
      f"IVF has {ivf.nlist} clusters with on average {N // ivf.nlist} vectors each.")

```

Output

python vector_foundations.py

```

=====
1. Curse of dimensionality: nearest and farthest points look alike
=====
  dims | nearest | farthest | farthest / nearest
    2 |   0.021 |   0.930 |         44.81
   10 |   0.434 |   1.988 |          4.58
  100 |   3.173 |   4.759 |          1.50
 1000 |  12.124 |  13.663 |          1.13
As dimensions grow the ratio tends to 1, so 'close' vs 'far' loses meaning and
range-based indexes (B-trees) cannot prune the search space.

=====
2. Text embeddings (dense vectors) for Task Hub task titles
=====
[ 0.59  0.81  0. ] Fix login bug in the authentication service
[ 0.73  0.68 -0. ] Add JWT token authentication to the login API
[ 0.58  0.81  0. ] Hash user passwords and secure the login token
[-0.   0.   1.] Design database schema for tasks and users
[-0.   0.   1.] Write SQL queries and indexes for the task database
[-0.   0.   1.] Normalise the database tables for users and projects
[ 0.73 -0.69  0. ] Create dark mode theme for the dashboard UI
[ 0.73 -0.69  0. ] Improve CSS layout of the mobile dashboard UI
[ 0.88 -0.48  0. ] Add charts to the project dashboard UI

Query: 'security issue with user login token' -> vector [0.605 0.796 0. ]
1. cosine=1.000 Fix login bug in the authentication service
2. cosine=1.000 Hash user passwords and secure the login token
3. cosine=0.986 Add JWT token authentication to the login API

=====
3. Similarity metrics: cosine, dot product, Euclidean distance
=====
 pair | cosine | dot | euclidean
  a,b |  1.000 | 28.0 |    3.742
  a,c |  0.714 | 10.0 |    2.828
cosine : direction only (a and b are identical in meaning) - use for text embeddings
dot     : direction AND magnitude - equals cosine when vectors are normalised
euclidean: straight-line distance - use when absolute magnitude matters

=====
4. Exact kNN vs ANN on 100,000 vectors of 128 dimensions (k = 10)
=====
Flat (exact brute force)      | build - | 2.958 ms/query | recall@10 = 1.000
IVF (nlist=256, nprobe=1)    | build 0.75s | 0.024 ms/query | recall@10 = 0.991
IVF (nlist=256, nprobe=8)    | build - | 0.173 ms/query | recall@10 = 1.000
IVF (nlist=256, nprobe=32)   | build - | 0.634 ms/query | recall@10 = 1.000
HNSW (M=32, efSearch=16)     | build 7.05s | 0.028 ms/query | recall@10 = 0.963
HNSW (M=32, efSearch=64)     | build - | 0.048 ms/query | recall@10 = 1.000
HNSW (M=32, efSearch=256)    | build - | 0.149 ms/query | recall@10 = 1.000

Trade-off: ANN indexes give up a little recall for a large speed-up. Increasing
nprobe (IVF) or efSearch (HNSW) raises recall at the cost of query time.
HNSW graph has 4 layers; IVF has 256 clusters with on average 390 vectors each.

```

Result

The farthest-to-nearest distance ratio fell from about 45 in 2 dimensions to 1.13 in 1000 dimensions, showing why range-based indexes cannot prune high-dimensional data. Dense text embeddings grouped security, database and UI tasks, and the login-token query retrieved the security tasks. Cosine treated vectors a and b as identical while dot product and Euclidean distance did not. On 100,000 vectors, IVF and HNSW answered queries up to about 100 times faster than brute force with recall@10 between 0.96 and 1.00, and higher nprobe/efSearch raised recall at the cost of query time.